



8-PUZZLE

PROYECTO FINAL

UNIVERSIDAD AUTÓNOMA DE AGUASCALIENTES

C.C.B I.S.C 4-B

PROGRAMACIÓN III

MTRO. EDUARDO SERNA PEREZ

AÍDA MONTSERRAT VENEGAS CONS

ID: 550894

VLADIMIR SUÁREZ VEGA

ID:548562

PAOLA SARAHI ZERMEÑO OJEDA

ID:551467

RESUMEN DESCRIPTIVO

Realizamos este proyecto con el propósito de desarrollar un videojuego interactivo basado en el clásico Puzzle de 8 piezas, integrando inteligencia artificial para que el propio sistema sea capaz de resolver el tablero de forma automática. La aplicación fue construida en Java usando la biblioteca Swing para la interfaz gráfica, y el núcleo de la solución automatizada se apoya en el algoritmo de búsqueda A* (A-estrella), que es uno de los algoritmos de búsqueda heurística más utilizados en inteligencia artificial.

El juego consiste en un tablero de 3×3 donde el jugador acomoda ocho fichas numeradas y un espacio vacío. El objetivo es ordenar las fichas del 1 al 8, dejando el espacio vacío en la última posición. La aplicación cuenta con tres niveles de dificultad (que varían la cantidad de movimientos aleatorios al mezclar el tablero), un sistema de puntuación que acumula los resultados del jugador en un archivo de texto, un modo de solución inteligente paso a paso, y una función de sugerencia de jugada para orientar al usuario cuando no sabe cómo continuar.

A lo largo del desarrollo identificamos tanto fortalezas como áreas de oportunidad en el proyecto. Como fortalezas destacamos que el algoritmo A* garantiza siempre la solución óptima gracias al uso de la distancia Manhattan como heurística admisible, y que la separación de responsabilidades entre clases hace el código mantenible y fácil de extender. El sistema de puntuaciones con persistencia en archivo también resultó una solución práctica y funcional sin depender de tecnologías externas. Por otro lado, como debilidades identificamos que el solver puede volverse lento o quedarse sin memoria con tableros muy desordenados, ya que A* en el peor caso explora un número exponencial de nodos; además, la interfaz gráfica está construida directamente sobre el formulario de NetBeans, lo que la hace poco flexible para cambios visuales futuros. Finalmente, el sistema de puntuaciones no valida duplicados por mayúsculas o minúsculas en el alias, lo que puede generar registros inconsistentes.

TEMAS INVESTIGADOS

Para poder desarrollar este proyecto fue necesario investigar y aplicar varios conceptos tanto de programación. A continuación, se describen los principales temas que se estudiaron:

Algoritmo A*:

El algoritmo A* es un método de búsqueda informada que encuentra el camino de menor costo entre un nodo inicial y un nodo meta dentro de un grafo. Su funcionamiento se basa en evaluar cada nodo candidato con la función $f(n) = g(n) + h(n)$, donde $g(n)$ representa el costo acumulado desde el nodo inicial hasta el nodo actual, y $h(n)$ es una función heurística que estima el costo restante para llegar a la meta. Para que A* encuentre siempre la solución óptima, la heurística debe ser admisible, es decir, nunca debe sobreestimar el costo real.

Heurística de distancia Manhattan:

Como heurística para guiar al algoritmo A* en el Puzzle-8, se implementó la distancia Manhattan. Esta consiste en sumar, para cada pieza del tablero, la cantidad de posiciones horizontales y verticales que la separan de su posición objetivo. Es una heurística admisible y consistente para este tipo de problema, lo que garantiza que A* produzca soluciones óptimas.

Solucionabilidad del Puzzle 8:

La interfaz de usuario se construyó con Java Swing utilizando el entorno NetBeans. Se emplearon componentes como JFrame, JButton, JPanel, JTextField y JOptionPane para crear una ventana de juego funcional y amigable. Los botones del tablero actúan directamente como las piezas del puzzle, y sus eventos se gestionan para detectar movimientos válidos y actualizar el estado del juego.

Interfaces Gráficas con Java Swing

La interfaz de usuario se construyó con Java Swing utilizando el entorno NetBeans. Se emplearon componentes como JFrame, JButton, JPanel, JTextField y JOptionPane para crear una ventana de juego funcional y amigable. Los botones del tablero actúan directamente como las piezas del puzzle, y sus eventos se gestionan para detectar movimientos válidos y actualizar el estado del juego.

Manejo de Archivos en Java

Para persistir las puntuaciones de los jugadores entre sesiones se investigó el uso de FileReader, FileWriter, BufferedReader y BufferedWriter. Los registros se almacenan en un archivo de texto plano (puntuaciones.txt) con formato separado por punto y coma, lo que permite leer y escribir de forma sencilla sin depender de bases de datos externas.

ESTRUCTURA Y FUNCIONAMIENTO

El proyecto está organizado en un solo paquete llamado `main_puzzle`, que contiene seis clases Java con responsabilidades bien diferenciadas. A continuación se describe cada una y cómo interactúan entre sí.

Clases del Proyecto

-Main_Puzzle.java

Es el punto de entrada de la aplicación. Su único método `main()` lanza el evento de creación de la ventana principal (`GraphicPuzzle`) usando el hilo de despacho de eventos de AWT, lo cual es la práctica recomendada para aplicaciones Swing.

-NodoPuzzle.java

Representa cada estado del tablero dentro del árbol de búsqueda. Almacena el arreglo bidimensional del tablero, una referencia al nodo padre (para reconstruir el camino solución), el valor `g` (movimientos desde el inicio), el valor `h` (heurística de Manhattan) y el valor `f = g + h`. Implementa la interfaz `Comparable` para que la `PriorityQueue` los ordene automáticamente por el valor `f` más bajo.

-SolverAEstrella.java

Es el núcleo inteligente del proyecto. Contiene dos métodos principales: `resolver()`, que ejecuta A* completo para encontrar la secuencia óptima de movimientos hasta llegar al estado meta, y `sugerirJugada()`, que aplica una variante del mismo algoritmo pero devuelve solamente el primer movimiento recomendado cuando se alcanza el límite de nodos o profundidad. Internamente incluye métodos auxiliares para generar sucesores (los estados alcanzables desde una posición deslizando una pieza adyacente al espacio vacío), calcular la distancia Manhattan, verificar si dos tableros son iguales, convertir el tablero a cadena de texto para la `HashSet`, copiar tableros y comprobar la solucionabilidad mediante conteo de inversiones.

-ResultadoBusqueda.java

Funciona como un objeto de transferencia de datos (DTO). Encapsula el resultado que devuelve el solver: la lista de tableros que forman el camino solución, la cantidad de nodos generados, la cantidad de nodos explorados, un booleano que indica si se encontró solución y un mensaje descriptivo del resultado. Esto permite que `GraphicPuzzle` procese el resultado de forma limpia sin conocer los detalles internos del solver.

-Puntuacion.java

Modelo simple que representa el registro de un jugador: alias, puntos acumulados y fecha de la última partida. Incluye el método sumarPuntos() que permite acumular los puntos de varias partidas bajo el mismo alias.

-ManejadorPuntuaciones.java

Se encarga de toda la lógica de persistencia. Lee y escribe el archivo puntuaciones.txt usando BufferedReader y BufferedWriter. Al guardar, busca si el alias ya existe en el archivo: si existe, acumula los nuevos puntos; si no, crea un nuevo registro. También expone un método para obtener las puntuaciones ordenadas de mayor a menor, que se muestra al jugador mediante un diálogo al finalizar la partida.

-GraphicPuzzle.java

Es la clase más extensa del proyecto. Extiende JFrame y gestiona toda la interfaz gráfica y la lógica del juego. Al inicializarse, mapea los nueve JButton del formulario a una matriz 3×3 y mezcla el tablero aleatoriamente. Los métodos más importantes son: moverFicha() (valida si el movimiento es legal y actualiza el tablero), mezclar() (genera una secuencia aleatoria de movimientos válidos según el nivel), resuelto() (verifica si el tablero está en el estado meta), calcularPuntos() (calcula la puntuación del nivel con la fórmula $1000 - \text{movimientos} \times 10$, con un mínimo de 100 puntos), guardarPuntuacionJugador() (solicita un alias al jugador mediante un diálogo y lo pasa al ManejadorPuntuaciones para persistirlo), obtenerFichaMovida() (compara el tablero actual con el siguiente estado para identificar qué pieza se desplazó al espacio vacío), resolverInteligente() (llama al solver, pide al usuario la profundidad máxima y el límite de nodos, y almacena el camino solución para su reproducción), mostrarSiguientePaso() (avanza un paso en la solución automática actualizando el tablero) y sugerirJugada() (pide al usuario una profundidad y muestra en el tablero el siguiente estado óptimo sugerido).

Flujo General del Juego

Al ejecutar la aplicación, el tablero se mezcla automáticamente con el nivel 1 (50 movimientos aleatorios). El jugador hace clic en las fichas adyacentes al espacio vacío para moverlas. Si completa el nivel, aparece un diálogo de felicitación y el juego avanza al siguiente nivel con mayor mezcla. Al completar los tres niveles, el sistema solicita un alias y guarda la puntuación total. En cualquier momento el jugador puede pedir una sugerencia o activar el modo solución inteligente, que recalcula A* desde el estado actual y muestra la ruta óptima paso a paso.

BITÁCORA DE TRABAJO

A continuación, se presenta un resumen de las etapas por las que pasó el desarrollo del proyecto, desde la planificación hasta las pruebas finales.

1 Planificación e investigación

Se analizaron los requerimientos del proyecto: un puzzle gráfico de 8 piezas con solución inteligente. Se investigó el funcionamiento del algoritmo A*, la heurística de distancia Manhattan y la condición de solucionabilidad mediante inversiones. También se revisaron ejemplos de implementación del puzzle en Java para entender las estructuras de datos necesarias.

2 Diseño de las clases

Se definió la arquitectura del paquete main_puzzle, separando responsabilidades: NodoPuzzle para el árbol de búsqueda, SolverAEstrella para la lógica IA, ResultadoBusqueda como DTO, Puntuacion y ManejadorPuntuaciones para el sistema de scores, y GraphicPuzzle para la interfaz. Este diseño facilitó trabajar en paralelo en distintas partes del sistema.

3 Implementación del solver A*

Se implementó SolverAEstrella con los métodos resolver() y sugerirJugada(). Se desarrollaron los auxiliares para generar sucesores válidos, calcular la distancia Manhattan, verificar igualdad de tableros y comprobar la solucionabilidad. Se realizaron pruebas aisladas del solver con tableros de ejemplo para validar que encontraba la solución óptima.

4 Desarrollo de la interfaz gráfica

Se construyó GraphicPuzzle.java con NetBeans, usando el diseñador visual de Swing para disponer los nueve botones del tablero y los controles de nivel, puntuación y acciones. Se conectaron los eventos de cada botón con la lógica de moverFicha(), y se implementó la lógica de mezcla, verificación de victoria y cambio de nivel.

5 Sistema de puntuaciones y persistencia

Se implementaron Puntuacion.java y ManejadorPuntuaciones.java. Se definió el formato del archivo puntuaciones.txt (alias;puntos;fecha) y se validó el comportamiento de acumulación de puntos cuando el mismo alias juega varias partidas. Se agregó la visualización del ranking en un diálogo al finalizar el juego.

6 Integración, ajustes y pruebas finales

Se integraron todos los componentes y se realizaron pruebas completas del flujo del juego: mezcla, movimientos, detección de victoria en los tres niveles, modo inteligente y sugerencia de jugada. Se

corrigieron detalles menores de interfaz y se verificó el funcionamiento del manejo de excepciones en la lectura/escritura del archivo de puntuaciones.

CONCLUSIONES

Este proyecto nos permitió entender de manera práctica cómo un algoritmo de inteligencia artificial puede aplicarse a un problema concreto de juego. Implementar A* desde cero hizo que comprendiéramos la importancia de elegir una buena heurística: la distancia Manhattan resulta ideal para el Puzzle-8 porque siempre subestima el costo real, lo que garantiza encontrar la solución óptima cuando existe.

También pudimos comprobar cómo las estructuras de datos correctas hacen una diferencia enorme en el rendimiento. Usar una PriorityQueue para la lista abierta y un HashSet para la lista cerrada permite que el solver maneje miles de nodos sin perder eficiencia, algo que no sería posible con listas simples.

En cuanto al diseño del software, aprendimos la importancia de separar responsabilidades entre clases, para poder mantener un flujo de trabajo más rápido y eficiente y procurar que el código sea mantenible y actualizable a futuro.

El sistema de puntuaciones con persistencia en archivo nos mostró cómo manejar datos entre sesiones sin necesidad de una base de datos, algo útil para proyectos de mediana escala. En general, el proyecto consolidó conceptos de Programación Orientada a Objetos, algoritmos de búsqueda y desarrollo de interfaces gráficas en un mismo producto funcional.

REFERENCIAS

1. DataCamp — Tutorial del Algoritmo A* — <https://www.datacamp.com/es/tutorial/a-star-algorithm>
2. Stack Overflow — Resolver el Puzzle-8 con A* en Java — <https://stackoverflow.com/questions/76094246/solve-8-puzzle-problem-using-a-algorithm-in-java>
3. Scribd — Proyecto 2: Puzzle de 8 piezas — <https://www.scribd.com/document/170480443/proyecto2>